PACK OPERATIVO — SAMUEL

Tech Lead · UGC Colombia

Fecha: 2026-04-09 **Versión:** 1.0 **Rol:** Arquitectura, stack Next.js + Supabase + n8n, integraciones, seguridad **Uso:** Referencia diaria para desarrollo, migraciones, setup y auditoría.

Tabla de contenidos

1. Arquitectura general
2. Variables de entorno
3. Schema Supabase (migración única)
4. Features AI (chatbot, dossier, briefs, match, reportes)
5. Integraciones API (8 proveedores)
6. Workflows n8n (10 críticos)
7. Hallazgos Security Audit + fixes pendientes
8. Runbook de deployment
9. KPIs y monitoreo

---

1. Arquitectura general

Stack:

- **Framework:** Next.js 15 App Router · Fluid Compute (no Edge) · AI SDK v6
- **BD:** Supabase (Postgres + pgvector) · RLS activo
- **IA:** Vercel AI Gateway con fallback `claude-sonnet-4.6 → gpt-5 → gemini-3.1-pro-preview`
- **Automatización:** n8n en `dev.kreoon.com` (VPS)
- **Hosting video:** Bunny CDN
- **Pagos:** Stripe (cliente) / Mercury + Wise (creadores)
- **Analytics:** Looker Studio + Supabase + Vercel Analytics
- **CI/CD:** Vercel (auto deploy main)

Principios de diseño:

- **Fluid Compute por default.** Todas las rutas AI corren en Node.js runtime. Fluid reutiliza instancias, baja cold starts, permite SDKs pesados.
- **AI Gateway como única puerta.** Cero claves directas en `process.env` de features producto. Tags por feature para cost tracking.
- **Supabase como source of truth.** Conversaciones, dossiers, embeddings, briefs, reportes. RLS activo.
- **AI SDK v6:** `streamText`, `generateText` con `Output.object()`, `ToolLoopAgent`, `InferAgentUIMessage`. NO usar `generateObject`, `maxSteps`, `parameters`, `maxTokens` (deprecados).
- **Modelo por tarea:**
  - Chat pre-sales / briefs / reportes → `anthropic/claude-sonnet-4.6`
  - Razonamiento profundo (dossier final, re-rank) → `anthropic/claude-opus-4.6`
  - Clasificación / routing barato → `google/gemini-3-flash`
  - Embeddings → `openai/text-embedding-3-small` (1536 dims)

Diagrama alto nivel:

```
 [ Cliente web ]──┐
                  ▼
        Next.js 15 App Router (Fluid Compute, Node runtime)
         ├─ /api/chat ───── ToolLoopAgent (Sonnet 4.6) ── Supabase / Cal.com
         ├─ /api/research/dossier ── Perplexity + Opus 4.6
         ├─ /api/briefs ─── generateText Output.object
         ├─ /api/match ───── pgvector search + re-rank
         └─ /api/reports ── Cron diario/mensual
                  │
                  ▼
        Supabase (Postgres + pgvector)
                  │
          ┌───────┴───────┐
          ▼               ▼
      n8n workflows  Bunny CDN
      (dev.kreoon)   (video hosting)
          │
          └── Stripe · Wise · Mercury · WhatsApp Cloud API · Cal.com
              Resend · Beehiiv · Meta Ads · Perplexity
```

---

2. Variables de entorno

```
# AI Gateway — OIDC automático en Vercel (sin API key manual)
# En dev local: `vercel env pull` y el SDK detecta el token.
# En producción: nada que definir, se rota solo.

# Supabase
NEXT_PUBLIC_SUPABASE_URL=
NEXT_PUBLIC_SUPABASE_ANON_KEY=
SUPABASE_SERVICE_ROLE_KEY=

# Research
PERPLEXITY_API_KEY=

# Booking
CAL_COM_API_KEY=
CAL_COM_EVENT_TYPE_ID=discovery-call-alexander
CAL_COM_WEBHOOK_SECRET=

# Cron
CRON_SECRET=

# WhatsApp Cloud API
WHATSAPP_ACCESS_TOKEN=
WHATSAPP_PHONE_NUMBER_ID=
WHATSAPP_WEBHOOK_VERIFY_TOKEN=
WHATSAPP_APP_SECRET=

# Meta Ads
META_ADS_ACCESS_TOKEN=
META_AD_ACCOUNT_ID=

# Stripe
STRIPE_SECRET_KEY=
STRIPE_WEBHOOK_SECRET=
```

```
# Resend (emails transaccionales)
RESEND_API_KEY=
RESEND_FROM_EMAIL=hello@ugccolombia.co

# Beehiiv (newsletter)
BEEHIIV_API_KEY=
BEEHIIV_PUBLICATION_ID=

# Wise + Mercury (pagos creadores)
WISE_API_TOKEN=
WISE_PROFILE_ID=
MERCURY_API_TOKEN=
MERCURY_ACCOUNT_ID=

# n8n webhooks (de Next.js hacia n8n)
N8N_WEBHOOK_BOOKING_CREATED=
N8N_WEBHOOK_PAYMENT_RECEIVED=
N8N_WEBHOOK_DELIVERY_APPROVED=
N8N_HMAC_SECRET=

# Bunny CDN
BUNNY_STORAGE_ZONE=
BUNNY_STORAGE_PASSWORD=
BUNNY_PULL_ZONE_URL=
```

Helper obligatorio (`lib/env.ts`):

```typescript
function requireEnv(key: string): string {
  const value = process.env[key];
  if (!value) throw new Error(`Missing required env var: ${key}`);
  return value;
}
export { requireEnv };
```

Regla: nunca usar `process.env.FOO!` (non-null assertion). Siempre `requireEnv('FOO')` .

---

3. Schema Supabase (migración única `ai_features.sql`)

```sql
-- Extensión
create extension if not exists vector;

-- Leads
create table if not exists public.leads (
  id uuid primary key default gen_random_uuid(),
  email text unique not null,
  full_name text,
  company text,
  website text,
  locale text default 'es',
  source text,
  bant jsonb default '{}'::jsonb,    -- { budget, authority, need, timing }
  stage text default 'new',          -- new | qualified | discovery_booked | client | lost
```

```sql
  created_at timestamptz default now()
);

-- Conversaciones chatbot
create table public.chat_conversations (
  id uuid primary key default gen_random_uuid(),
  lead_id uuid references public.leads(id) on delete cascade,
  locale text default 'es',
  status text default 'active',      -- active | qualified | handoff
  summary text,
  created_at timestamptz default now(),
  updated_at timestamptz default now()
);

create table public.chat_messages (
  id uuid primary key default gen_random_uuid(),
  conversation_id uuid references public.chat_conversations(id) on delete cascade,
  role text not null,                -- user | assistant | tool
  parts jsonb not null,              -- UIMessage parts (AI SDK v6)
  tokens_in int,
  tokens_out int,
  cost_usd numeric(10,6),
  created_at timestamptz default now()
);
create index on public.chat_messages (conversation_id, created_at);

-- Dossiers (brand research)
create table public.brand_dossiers (
  id uuid primary key default gen_random_uuid(),
  lead_id uuid references public.leads(id) on delete cascade,
  company text not null,
  industry text,
  hero_product text,
  competitors jsonb,                 -- [{name, url, note}]
  ugc_opportunities jsonb,           -- [{angle, format, rationale}]
  sources jsonb,                     -- Perplexity citations
  raw_markdown text,
  status text default 'pending',     -- pending | ready | failed
  created_at timestamptz default now()
);

-- Briefs generados
create table public.briefs (
  id uuid primary key default gen_random_uuid(),
  client_id uuid,
  product text not null,
  audience text not null,
  objective text not null,
  format text,                       -- testimonial | unboxing | how-to | ...
  content jsonb not null,            -- brief estructurado
  markdown text,
  created_at timestamptz default now()
);

-- Pool de creadores + embeddings
create table public.creators (
  id uuid primary key default gen_random_uuid(),
  full_name text not null,
  handle text,
  tier text check (tier in ('A','B','C')),
```

```
  niches text[],
  city text,
  rate_usd numeric,
  availability boolean default true,
  bio text,
  portfolio_urls text[],
  embedding vector(1536),            -- text-embedding-3-small
  created_at timestamptz default now()
);
create index on public.creators using hnsw (embedding vector_cosine_ops);
create index on public.creators using gin (niches);

-- Match results (auditoría)
create table public.creator_matches (
  id uuid primary key default gen_random_uuid(),
  brief_id uuid references public.briefs(id) on delete cascade,
  creator_id uuid references public.creators(id) on delete cascade,
  score numeric,
  rationale text,
  rank int,
  created_at timestamptz default now()
);

-- Reportes mensuales
create table public.client_reports (
  id uuid primary key default gen_random_uuid(),
  client_id uuid not null,
  period date not null,              -- primer día del mes
  metrics jsonb not null,
  narrative_md text,
  pdf_url text,
  generated_at timestamptz default now(),
  unique (client_id, period)
);

-- Tablas operativas adicionales (crear en migración #2)
create table public.clients (
  id uuid primary key default gen_random_uuid(),
  lead_id uuid references public.leads(id),
  company text not null,
  plan text check (plan in ('starter','pro','premium')),
  billing_day int,
  status text default 'active',  -- active | suspended | churned
  stage text default 'onboarding', -- onboarding | recurring
  briefing_approved_at timestamptz,
  strategy_approved_at timestamptz,
  created_at timestamptz default now()
);

create table public.deliveries (
  id uuid primary key default gen_random_uuid(),
  client_id uuid references public.clients(id),
  creator_id uuid references public.creators(id),
  brief_id uuid references public.briefs(id),
  status text default 'briefed', -- briefed | raw_delivered | approved | rejected
  raw_url text,
  final_url text,
  quality_grade text check (quality_grade in ('A','B','C')),
  created_at timestamptz default now()
);
```

```sql
create table public.creator_payments (
  id uuid primary key default gen_random_uuid(),
  delivery_id uuid references public.deliveries(id),
  creator_id uuid references public.creators(id),
  type text check (type in ('advance','final')),
  amount numeric not null,
  method text,
  external_ref text,
  idempotency_key text unique not null,
  status text default 'pending', -- pending | initiating | submitted | sent | failed
  provider_tx_id text,
  paid_at timestamptz,
  created_at timestamptz default now()
);

create table public.payments (
  id uuid primary key default gen_random_uuid(),
  client_id uuid references public.clients(id),
  stripe_invoice_id text unique,
  amount_usd numeric not null,
  status text default 'pending', -- pending | paid | failed
  mrr_paid_at timestamptz,
  created_at timestamptz default now()
);
```

RLS: `chat_conversations` y `chat_messages` abiertos a `anon` solo para insert con `conversation_id` coincidente (JWT custom claim). Resto solo `service_role`.

---

4. Features AI

Feature 1 — Chatbot pre-sales `/chat`

Runtime: Node.js / Fluid Compute. `maxDuration = 60`.

```
Client <useChat> ─► POST /api/chat ─► ToolLoopAgent (Sonnet 4.6)
    ▲                │                    │
    │                │                    ├─► getLeadHistory ─► Supabase
    │                │                    ├─► upsertLead ────► Supabase
    │                │                    ├─► saveBant ──────► Supabase
    │                │                    ├─► getAvailableSlots ─► Cal.com
    │                │                    └─► bookDiscoveryCall ─► Cal.com + trigger dossier
    │                │
    └── stream UI ───┘
            │
            └─► persist user+assistant → chat_messages
```

Handoff: BANT completo (`budget ≥ 1500 USD && authority && timing < 90d`) → `bookDiscoveryCall` dispara además `/api/research/dossier`.

System prompt base (ES/EN bilingüe):

```
 Eres Lía, la consultora virtual de UGC Colombia — agencia boutique de contenido
generado por creadores premium en LATAM. Tu rol es calificar leads antes del
discovery call con Alexander Cast (founder).
```

```
IDIOMA: Detecta el idioma del lead en su primer mensaje. Responde siempre en
ese idioma (español o inglés). Nunca mezcles.

TONO: Cálida, profesional, concreta. Nada de emojis. Frases cortas. Nunca
prometas precios en el chat — eso se cierra en el discovery call.

MISIÓN: En 6-10 turnos, descubrir:
  1. BUDGET — rango de inversión mensual en UGC (USD).
  2. AUTHORITY — rol del lead y quién firma.
  3. NEED — qué problema de contenido/performance están resolviendo.
  4. TIMING — cuándo quieren empezar.

REGLAS:
- Usa getLeadHistory ANTES de preguntar datos que ya tengamos.
- Guarda cada señal con saveBant a medida que la captures.
- Cuando BANT esté completo y califique (budget ≥ 1500 USD, timing ≤ 90 días),
  ofrece agendar discovery call con getAvailableSlots + bookDiscoveryCall.
- Si NO califica, agradece y ofrece recursos gratuitos (lead magnet PDF).
- Nunca inventes casos de estudio ni cifras. Si no sabes, dilo.
- Nunca digas que eres IA a menos que pregunten directamente.
```

Feature 2 — Research Dossier `/api/research/dossier`

Trigger: post-booking de discovery call. Usa Perplexity para research + Opus 4.6 para síntesis. Output estructurado con `Output.object()`: company, industry, hero_product, competitors, ugc_opportunities, sources. Persiste en `brand_dossiers`.

Feature 3 — Brief Generator `/api/briefs`

Input: client_id, product, audience, objective, format. Genera brief estructurado con Sonnet 4.6 + `Output.object()`. Persiste en `briefs` con markdown + jsonb.

Feature 4 — Creator Match `/api/match`

1. Embed del brief con `text-embedding-3-small`.
2. `pgvector` HNSW cosine search sobre `creators.embedding`.
3. Filtro por `niches && ARRAY[:tags]::text[]` + `availability=true` + `tier >= :min_tier`.
4. Top 20 → re-rank con Opus 4.6 (razonamiento sobre bio + portfolio).
5. Top 5 guardados en `creator_matches` con score + rationale + rank.

Feature 5 — Reportes mensuales `/api/reports/monthly`

Cron mensual (día 1, 6am COT). Por cada `clients.status='active'`:

1. Pull métricas de Meta Ads + datos de `deliveries` del mes.
2. Genera narrativa con Sonnet 4.6.
3. Persiste en `client_reports` (unique `client_id, period`).
4. Genera PDF con `@react-pdf/renderer` → sube a Bunny CDN.
5. Dispara webhook n8n → WhatsApp al cliente con link PDF.

---

5. Integraciones API

Stack compartido: `lib/integrations/{provider}-client.ts` con helper `XFetch<T>` con retry + backoff + timeout por intento.

Convenciones globales:

- MAX_RETRIES = 3, BASE_DELAY_MS = 1000, backoff exponencial.
- Retryable: 429, 500, 502, 503, 504.
- Timeout livianos: 10s. Pesados: 30s. Webhooks entrantes: responder <5s.
- Headers: `Content-Type: application/json`, `Authorization`, `User-Agent: ugccolombia-app/1.0`.

Proveedores integrados

#	Proveedor	Caso de uso	Endpoints clave
1	**Beehiiv**	Newsletter: suscribir, crear/publicar posts, leer stats	`/v2/publications/{id}/subscriptions`, `/posts`, `/posts/{id}/stats`
2	**Resend**	Emails transaccionales: bienvenida, recordatorios, reportes	`/emails`, `/emails/batch`
3	**Cal.com**	Booking discovery call, webhooks BOOKING_CREATED	`/v2/slots`, `/v2/bookings`. Webhook HMAC sha256
4	**WhatsApp Cloud API**	Mensajes, templates, media, webhooks entrantes	`/v17.0/{phone_id}/messages`. Webhook HMAC X-Hub-Signature-256
5	**Meta Ads**	Métricas de ads para reportes mensuales	`/v17.0/act_{id}/insights`
6	**Wise**	Transferencias internacionales a creadores	`/v1/profiles/{id}/transfers`. **Idempotency-Key obligatorio**
7	**Mercury**	Payouts en USD para creadores USA	`/api/v1/payment`. Idempotency-Key obligatorio
8	**Perplexity**	Research de mercado y brand dossier	`/chat/completions` model `sonar-large`

Ejemplo — Beehiiv Fetch con retry correcto (post-fix H1/H2 security audit):

```
async function beehiivFetch<T>(path: string, options: RequestInit = {}): Promise<T> {
  const apiKey = requireEnv('BEEHIIV_API_KEY');
  const headers = {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${apiKey}`,
    'User-Agent': 'ugccolombia-app/1.0',
    ...options.headers,
  };

  for (let attempt = 0; attempt < MAX_RETRIES; attempt++) {
    const controller = new AbortController();
    const timeoutId = setTimeout(() => controller.abort(), 10_000);
    try {
      const response = await fetch(`${BASE_URL}${path}`, { ...options, headers, signal: controller.signal });
      if (response.ok) return await response.json() as T;
      if (RETRYABLE_STATUSES.includes(response.status)) {
        const delay = BASE_DELAY_MS * Math.pow(2, attempt);
        console.warn(`[Beehiiv] ${response.status} en ${path}, reintento ${attempt + 1} en ${delay}ms`);
        await new Promise(r => setTimeout(r, delay));
        continue;
      }
      const body = await response.text();
      throw new Error(`[Beehiiv] ${response.status} ${response.statusText}: ${body}`);
    } catch (error) {
      console.error(`[Beehiiv] Intento ${attempt + 1}/${MAX_RETRIES} falló en ${path}:`, error);
      const isAbort = error instanceof Error && error.name === 'AbortError';
      const isNetwork = error instanceof TypeError;
      if (!isAbort && !isNetwork) throw error; // bugs → no reintentar
```

```
      if (attempt === MAX_RETRIES - 1) throw error;
    } finally {
      clearTimeout(timeoutId);
    }
  }
  throw new Error(`[Beehiiv] Max reintentos alcanzados para ${path}`);
}
```

Validación de webhooks (HMAC timingSafeEqual obligatorio):

```
import { createHmac, timingSafeEqual } from 'crypto';

export function verifyCalWebhook(rawBody: string, signature: string): boolean {
  const secret = requireEnv('CAL_COM_WEBHOOK_SECRET');
  const expected = createHmac('sha256', secret).update(rawBody).digest();
  const received = Buffer.from(signature.replace(/^sha256=/, ''), 'hex');
  if (expected.length !== received.length) return false;
  return timingSafeEqual(expected, received);
}
```

---

6. Workflows n8n (10 críticos)

Corren en `dev.kreoon.com` (VPS). Endpoints HTTP protegidos con HMAC compartido (`N8N_HMAC_SECRET`).

#	Workflow	Trigger	Descripción
1	`lead-capture-to-crm`	Webhook (landing form)	Upsert `leads` , inicia chat Jarvis, envía welcome Resend
2	`chat-handoff-to-alexander`	Supabase `chat_conversations.status='handoff'`	WhatsApp a Alexander con contexto + link a conversación
3	`discovery-call-booked`	Cal.com webhook BOOKING_CREATED	Persiste booking, dispara `research-dossier` , envía confirmación WhatsApp
4	`stripe-invoice-paid`	Stripe webhook `invoice.paid`	Activa cliente en Supabase, handoff Diana, welcome WhatsApp via Jarvis
5	`stripe-invoice-failed`	Stripe webhook `invoice.payment_failed`	Reintentos + dunning escalonado (ver Pack Brian)
6	`client-report-monthly`	Cron día 1 6am COT	Para cada cliente activo: pull Meta Ads + genera reporte + PDF + WhatsApp
7	**`creator-payment-scheduler`**	Manual + cron	**Crítico: pagar creadores con idempotencia (ver hallazgo H3)**
8	`delivery-deadline-reminder`	Cron diario 9am	Alertas de entregas pendientes a creadores + Diana
9	`whatsapp-incoming-router`	WhatsApp Cloud API webhook	Dispatch por intent: ventas → Jarvis, operaciones → Diana, creator → Diana

10	`weekly-report-to-team`	Cron viernes 4pm COT	Resumen semanal automático a todo el equipo (MRR, nuevos clientes, KPIs)

Workflow 7 — creator-payment-scheduler (flujo corregido post-H3)

```
 [Trigger: delivery.approved + client paid]
  ↓
1. Calcular idempotency_key = sha256(creator_id|delivery_id|type)
  ↓
2. INSERT creator_payments (status='initiating', idempotency_key) -- unique constraint previene doble pago
  ↓
3. POST al proveedor (Wise/Mercury) con header Idempotency-Key
  ↓
4. Verificar response.ok + body structure (Wise: status + id | Mercury: code 201)
  ↓
5. UPDATE creator_payments SET status='submitted', provider_tx_id=response.id
  ↓
6. Polling o webhook del banco → UPDATE status='sent' o 'failed'
  ↓
7. Solo cuando status='sent':
   - Marcar content_pieces.paid=true
   - Notificar al creador por WhatsApp
  ↓
8. Error branch → WhatsApp a Brian + Alexander con {creator_id, amount, tx_id, error}
```

---

7. Hallazgos Security Audit + fixes pendientes

Auditor: silent-failure-hunter. **Fecha audit:** 2026-04-08.

HIGH (bloquean producción)

H1. Timeout de AbortController creado UNA sola vez fuera del loop de reintentos

Afecta: beehiiv, resend, cal, whatsapp, meta, wise, mercury, perplexity (8 clientes). **Problema:** el timeout total es 10s para toda la operación, no 10s por intento. En el segundo intento el signal ya está abortado. `clearTimeout` solo se llama en `response.ok` → leak en 4xx. **Fix:** mover `new AbortController()` + `setTimeout` **dentro** del loop + `clearTimeout` en `finally`. **Estado: PENDIENTE** — aplicar patrón de sección 5 a los 8 clientes.

H2. Catch genérico traga errores distintos a AbortError sin loggear

Problema: `TypeError` (DNS, red), `SyntaxError` de `.json()`, bugs de código, todos caen al continue silencioso. Debugging imposible. **Fix:** loggear cada intento, distinguir `AbortError` vs `TypeError` vs otros. Errores no-red/no-abort = throw inmediato (bug, no reintentar). **Estado: PENDIENTE** — aplicar a los 8 clientes.

H3. `creator-payment-scheduler` sin rollback ni verificación de éxito real del pago

Problema: payments.status='scheduled' antes de confirmar banco, marca paid=true sin verificar, notifica al creador antes de confirmación, sin idempotencia robusta. **Riesgo de pagos duplicados o perdidos. Fix:** ver flujo corregido en sección 6 workflow 7 (idempotency_key, Idempotency-Key header, status progresivo initiating→submitted→sent, notificación solo en sent, error branch obligatorio). **Estado: CRÍTICO — PENDIENTE**. Bloquea primeros pagos a creadores.

H4. Validación HMAC con comparación `===` (timing attack)

Afecta: cal webhook, whatsapp webhook. **Fix:** usar `crypto.timingSafeEqual` (ver sección 5). Además loggear IP + signature en rechazos 401. **Estado: PENDIENTE**.

H5. Cal.com webhook delega a n8n sin verificar respuesta ni timeout

Problema: `process.env.N8N_WEBHOOK_BOOKING_CREATED!` sin validación, sin try/catch, sin timeout, sin verificar `response.ok`. Si n8n cae, el booking se pierde silenciosamente. **Fix:** `requireEnv`, try/catch, timeout 3s (webhook debe responder <5s), verificar response.ok, loggear éxito y fallo. **Estado: PENDIENTE**.

MEDIUM (revisar antes de scale-up)

- **M1.** Parseo de respuestas de Claude sin validación Zod → riesgo de runtime errors.
- **M2.** Scoring BANT en Supabase sin transaction (race condition posible si Jarvis + web concurrente).
- **M3.** Cron de reportes mensual sin lock distribuido → posible ejecución doble.
- **M4.** RLS abierto en `chat_messages` para insert desde `anon` sin rate limiting.
- **M5.** Falta `@upstash/ratelimit` en formulario público de suscripción Beehiiv.

LOW (mejoras de DX)

- **L1.** Falta `lib/env.ts` centralizado (hoy cada cliente reimplementa `requireEnv`).
- **L2.** No hay test suite de integraciones (mocks de fetch + Zod schemas).
- **L3.** `User-Agent` no incluye versión dinámica (hardcoded `1.0`).
- **L4.** Logs no estructurados (usar `pino` para JSON logs en producción).

Checklist pre-producción

- [ ] H1 aplicado a 8 clientes
- [ ] H2 aplicado a 8 clientes
- [ ] H3 workflow 7 refactorizado con idempotencia
- [ ] H4 HMAC timingSafeEqual en cal + whatsapp
- [ ] H5 cal webhook con try/catch + timeout + validación env
- [ ] Tests e2e del flujo chatbot → BANT → booking → dossier
- [ ] Smoke test de pago real a creador de prueba
- [ ] Dry-run de workflows n8n con datos sintéticos
- [ ] RLS audit con `supabase db lint`

---

8. Runbook de deployment

Setup inicial

```
# Clonar
git clone https://github.com/AlexanderKast/ugc-colombia.git
cd ugc-colombia

# Variables
cp .env.example .env.local
# Completar .env.local con las credenciales del equipo

# Instalar
pnpm install

# Migraciones Supabase
```

```
supabase db push

# Seed (opcional, para desarrollo)
pnpm run seed
```

Desarrollo local

```
pnpm dev                  # Next.js con Turbopack
vercel env pull           # Sincronizar env desde Vercel (incluye AI Gateway OIDC)
```

Deploy

Automático: push a `main` → Vercel auto-deploy. **Manual:** `vercel --prod` desde el directorio.

Rollback

```
vercel rollback            # al último deployment estable
# o
vercel promote <deployment-url>
```

Migraciones

```
supabase migration new <name>
# editar archivo en supabase/migrations/
supabase db push           # aplicar local
supabase db push --linked  # aplicar producción
```

---

9. KPIs y monitoreo

KPIs propios Samuel

KPI	Target
Uptime stack (web, Supabase, Jarvis)	≥99.5%
Horas operativas ahorradas/mes	≥40h
Bugs críticos abiertos	0
Time-to-recovery (incidente crítico)	<15 min
% procesos automatizados vs manuales	≥70% en M3

Cadencias

- **Diario:** uptime check (Vercel + Supabase + Jarvis), bugs críticos abiertos
- **Semanal:** tickets cerrados, automatizaciones nuevas activas, integraciones rotas

- **Mensual:** % procesos automatizados, ahorro horas, roadmap KREOON↔UGC

Herramientas de observabilidad

- **Vercel Analytics** — requests, latencia, errores Next.js
- **Supabase Dashboard** — queries lentas, connection pool, RLS violations
- **n8n Executions** — workflows activos, fallos, tiempo de ejecución
- **Upstash / Sentry (pendiente)** — error tracking estructurado
- **Looker Studio** — dashboards cliente-facing

Alertas críticas (WhatsApp a Samuel)

1. Cualquier workflow n8n que falle 3 veces consecutivas
2. Stripe webhook signature inválida (posible ataque)
3. `creator_payments.status='failed'` más de 1 por día
4. Supabase >90% connection pool
5. `/api/chat` latencia p95 >10s
6. Cualquier integración API con error rate >5% en 1h

Apéndice — Sinergia KREOON ↔ UGC Colombia

- **UGC Colombia es el cliente cero de KREOON.** Samuel construye el módulo "Briefs y Creadores" en KREOON usando UGC Colombia como cliente real.
- **Action Q2:** módulo debe estar operativo para M3 (junio 30).
- **Compartir schema:** tablas `creators`, `briefs`, `deliveries`, `creator_matches` son el core del módulo KREOON. Migraciones en `ai_features.sql` son la base.

Fuentes: `content/sistemas/ai-features/ai-architecture.md`, `content/sistemas/integraciones/api-integrations-spec.md`, `content/sistemas/qa/silent-failures-report.md`, `content/sistemas/qa/vercel-performance-plan.md`, `content/sistemas/qa/type-design-analysis.md`, `n8n/workflows-spec.md`.

Próxima revisión: 2026-05-09